

AWS Data Engineer Track — Guided Capstone Project
RetailFlow Lakehouse: From Raw Files to BI-Ready Analytics
Requirements & Task Document

1. Project Overview

RetailFlow is a fictional mid-size e-commerce company. You have been brought on as the Data Engineer responsible for building their first cloud data platform. Over the course of this capstone you will take raw, messy order data all the way through a governed data lake, into two parallel analytics engines (Amazon Redshift and Databricks), and out to business-ready dashboards.

1.1 Glossary — Key Terms You'll See

Term	Plain-English meaning
Bronze / Silver / Gold	Three quality stages of the same data: Bronze = raw and untouched, Silver = cleaned, Gold = summarized and ready for a dashboard.
Partition	A folder-like split of a dataset (e.g. by date) so queries only have to read the slice they need instead of everything.
ETL	Extract, Transform, Load — the process of moving data from a source, cleaning/reshaping it, and writing it somewhere new.
Idempotent	Safe to run twice. Re-running an idempotent job does not create duplicate data.
Data quality rule	An automated check (e.g. "this column can't be empty") that runs as part of the pipeline instead of being checked by hand.
Star schema	A simple table layout for reporting: one big "fact" table of transactions surrounded by smaller "dimension" tables that describe who/what/when.
Data lake / warehouse / lakehouse	A data lake stores raw files cheaply; a warehouse (like Redshift) is optimized for fast SQL reporting; a lakehouse (like Databricks/Delta) tries to do both.
Medallion architecture	Databricks' name for the same Bronze/Silver/Gold pattern used in the S3 data lake.

1.2 Learning Objectives

- Write a well-organized Python/Boto3 script to upload files to S3, with basic logging and retry logic.
- Design a simple three-zone (Bronze/Silver/Gold) S3 data lake with sensible folder naming, basic security, and a lifecycle rule.
- Get a first hands-on look at access governance with AWS Lake Formation and LF-Tags.
- Build a beginner-friendly AWS Glue ETL job that cleans data, re-runs safely using Job Bookmarks, and checks data quality automatically.
- Model and load a simple star schema in Amazon Redshift Serverless, and see how Redshift Spectrum queries S3 data directly.
- Build a basic Delta Lake medallion pipeline on Databricks using Auto Loader and Delta Live Tables, governed by Unity Catalog.
- Present your results through a simple SQL dashboard on both engines.

1.3 Environments You Will Use

Environment	What you'll do there
AWS Free Tier account	S3, IAM, Glue, Lake Formation, Athena, Redshift Serverless, CloudWatch
VS Code (local machine)	Python project, Boto3 scripts, Glue ETL script authoring, git repo, AWS CLI
Google Colab	Synthetic data generation, Pandas/NumPy profiling, Matplotlib/Seaborn charts
Databricks Free Trial workspace	Delta Lake, Auto Loader, Delta Live Tables, Unity Catalog, Workflows, SQL Warehouse

Note on Databricks: the Community Edition does not support Unity Catalog, Auto Loader file-notification mode, or Delta Live Tables in the way this capstone requires. Sign up for the Databricks free trial (14 days, credit-funded, full Premium feature set on your own AWS account) instead of Community Edition.

1.4 Cost & Safety Guardrails (read before you start)

- Create a Budget + Budget Alert in AWS Billing for \$5 the moment you create your account.

- Use Redshift Serverless (not a provisioned cluster) and set a low base RPU (8) with auto-pause; delete/pause the workgroup at the end of each work session.
- Use Glue worker type G.1X with 2–3 workers max, and only run jobs on demand — never on a tight schedule during development.
- Keep all datasets small (tens of thousands of rows, not millions) — this is a patterns exercise, not a scale exercise.
- Tear down (delete) Redshift workgroups/namespaces, Glue endpoints, and Databricks clusters at the end of each phase if you are not continuing immediately.

2. The Dataset

You will generate a synthetic RetailFlow dataset using Python's Faker library — this avoids licensing issues and lets you control size, dirtiness, and schema drift precisely.

2.1 Entities

File	Grain	Approx. rows	Notes
customers.csv	1 row per customer	5,000	Includes a mix of valid/missing emails, some duplicate customer_ids to simulate dirty data
products.csv	1 row per product	800	category, unit_price, active_flag
orders_day1.json	1 row per order header	20,000	order_id, customer_id, order_ts, store_region, status
order_items_day1.json	1 row per order line	~55,000	order_id, product_id, quantity, unit_price, line_total
orders_day2.json / order_items_day2.json	Next day's incremental batch	~4,000 / ~11,000	Adds one new column, discount_code, that did not exist in day1 — this simulates a very common real-world event called schema evolution
clickstream_day1.json, clickstream_day2.json	Web session events	~15,000/day	Used only in the Databricks Auto Loader phase to demonstrate streaming-style ingestion

2.2 Deliberate Data Quality Issues to Inject

- ~2% of customers.csv rows with null or malformed email addresses.
- ~1% duplicate customer_id values across rows.
- A handful of order_items rows referencing a product_id that does not exist in products.csv (referential integrity failures).

- A handful of negative or zero quantity values.
- day2 files introduce a new discount_code column not present in day1 (schema evolution test).

3. Project Phases, Tasks & Deliverables

Complete the phases in order — each phase builds on the artifacts of the previous one. Deliverables for every phase must be committed to a git repository named retailflow-capstone.

Phase 0 — Environment Setup

1. Create/verify your AWS free tier account; enable MFA on the root user; create an IAM admin user for daily use.
2. Set a \$5 AWS Budget with an email alert at 50%/80%/100%.
3. Install AWS CLI, Python 3.11+, and the AWS Toolkit / Python extensions in VS Code; run aws configure.
4. Create a Google Colab notebook named 01_data_generation.ipynb.
5. Sign up for the Databricks free trial, linked to your AWS account (or Databricks-managed) and confirm you can create a cluster.

Deliverable: screenshot of AWS Budget, confirmation Databricks workspace is active, git repo initialized with a README.

Phase 1 — Synthetic Data Generation & Profiling (Colab)

6. In Colab, use Faker + NumPy to generate all files listed in Section 2, including the deliberate data-quality issues and the schema-evolution scenario.
7. Profile every file with Pandas: null rates per column, dtype summary, duplicate counts, value ranges.
8. Produce at least 4 charts with Matplotlib/Seaborn: order volume by day, revenue distribution, top 10 categories by revenue, and a null-rate heatmap across all files.
9. Save all generated files locally and export the charts as PNGs.

Deliverable: 01_data_generation.ipynb (exported to the repo), a data_profile_report.md summarizing findings, 4+ PNG charts.

Phase 2 — Boto3 Ingestion Utility (VS Code)

10. Build a Python package s3_ingest/ with a reusable Boto3 client wrapper: upload_file, list_objects, download_file, and a presigned-URL generator.
11. Implement structured logging (the logging module, not print) and exponential-backoff retry logic around every S3 call.

12. Read AWS credentials only via environment variables or a shared credentials profile — never hard-code keys. Add a .env.example and a requirements.txt.
13. Use the utility to upload the day1 files to s3://<your-bucket>/raw/ under a partitioned key structure (see Phase 3).

Deliverable: s3_ingest/ package with unit tests for the retry logic, upload_day1.py script, log output showing at least one simulated retry.

Phase 3 — S3 Data Lake Foundations

14. Create one S3 bucket with three top-level prefixes: raw/ (Bronze), curated/ (Silver), consumption/ (Gold).
15. Partition raw/ by dataset name and ingestion date, e.g. raw/orders/dt=2026-07-06/.
16. Enable versioning and default SSE-S3 (or SSE-KMS) encryption on the bucket.
17. Attach a lifecycle policy: transition raw/ objects to Standard-IA after 30 days and add a rule to abort incomplete multipart uploads after 7 days.
18. Configure an S3 event notification on raw/ that would trigger a Glue job/Lambda on new object arrival (EventBridge rule is sufficient — you do not have to wire a live trigger for this capstone, but the rule must exist and be documented).
19. Write a one-page ARCHITECTURE.md describing the three zones, what belongs in each, and your partitioning/naming convention.

Deliverable: screenshots of bucket structure, versioning/encryption settings, lifecycle rule, and ARCHITECTURE.md.

Phase 4 — Glue Data Catalog, Crawlers & Athena

20. Create a Glue database retailflow_raw and run a Crawler over raw/ to populate the Data Catalog.
21. Fix at least one schema issue the crawler introduces (e.g. incorrect type inference on a partition column).
22. Set up partition projection on the orders table as an alternative to crawler-based partition discovery; document the trade-off.
23. Run at least 3 Athena queries against the raw tables and record the bytes scanned for each — once against the un-pruned table and once using a WHERE clause on the partition column, to show the cost impact of partition pruning.

Deliverable: athena_cost_comparison.md with query text + bytes-scanned screenshots for pruned vs. unpruned queries.

Phase 5 — Lake Formation Governance

24. Register your S3 bucket location in Lake Formation and switch the retailflow_raw database to Lake-Formation-managed permissions.
25. Create LF-Tags: sensitivity = {PII, Confidential, Public} and department = {analytics, engineering}.
26. Tag the customers table's email column as PII, and tag the curated/gold tables (once they exist in Phase 6) as Confidential.
27. Create two IAM personas/roles — data_analyst and data_engineer — and grant each different table/column-level permissions via LF-Tag expressions.
28. Prove the access boundary: query the same table via Athena while assuming each role and show the analyst role cannot see the email column.

Deliverable: lake_formation_governance.md with the LF-Tag assignments and the two access-proof screenshots.

Phase 6 — Glue ETL: Raw → Silver, with Data Quality Gates

This phase introduces the two most important habits in real-world data engineering: making a job safe to re-run (idempotency) and catching bad data automatically instead of by hand.

29. Author a Glue PySpark ETL job (in a Glue Studio notebook or local Glue Docker image, developed in VS Code) that reads raw orders/order_items/customers/products, applies type casting, null handling, and deduplication, and writes partitioned Parquet to curated/.
30. Enable Job Bookmarks so re-running the job only processes new data. Run the job against day1, then again after adding day2 files, and show in your write-up that only day2 rows were processed the second time.
31. Handle the day2 schema change (discount_code) using mergeSchema semantics without breaking the day1-shaped rows.
32. Define AWS Glue Data Quality rules in DQDL covering: Completeness on required columns, Uniqueness on order_id, and Referential Integrity between order_items.product_id and products.product_id.

33.Route rows that fail data quality to a quarantine/ prefix instead of curated/, and publish the quality score to CloudWatch.

Deliverable: glue_etl_job.py, the DQDL ruleset file, a bookmarks_proof.md (before/after row counts), and a screenshot of the CloudWatch quality metric.

Phase 7 — Amazon Redshift Serverless Analytics Track

Here you move from the data lake into a proper reporting database. Redshift is where a business analyst would actually run their dashboards from.

34.Provision a Redshift Serverless workgroup/namespace (base 8 RPU, auto-pause enabled).

35.Design and create a simple star schema: dim_customer, dim_product, dim_date, and fact_order_items, with explicit distribution styles and sort keys.

36.Load the dimension and fact tables from curated/ using the COPY command with a manifest file; demonstrate error handling using STL_LOAD_ERRORS.

37.Create an external schema over consumption/ (Gold) and query it live via Redshift Spectrum.

38.Create at least one materialized view answering a realistic business question (e.g. daily revenue by category) with auto-refresh enabled.

39.Read about Workload Management (WLM) and Query Monitoring Rules, and configure one simple rule that aborts queries running longer than a threshold.

40.Apply a dynamic data masking policy on a PII column and show it is masked for one role and visible for another.

41.Capture one EXPLAIN plan and write 2–3 sentences describing what it tells you about how the query ran.

Deliverable: redshift_ddl.sql, redshift_load.sql, redshift_governance.sql, and a redshift_performance.md with the annotated EXPLAIN plan.

Phase 8 — Databricks Lakehouse Track

This phase repeats the Bronze/Silver/Gold idea from Phase 3, but on Databricks using Delta Lake instead of plain S3 files — this is a very common alternative architecture you'll see in the industry.

42.Create an all-purpose cluster (small, single-node, autoscaling off) and a Databricks Repo connected to your git repository.

43. Build Bronze Delta tables from the curated/ S3 data (external location + storage credential) — no instance-profile shortcuts.
44. Use Auto Loader in directory-listing mode to incrementally ingest the clickstream_day1/day2 JSON files into a Bronze Delta table, with checkpointing.
45. Build Silver (cleansed/deduplicated/conformed) and Gold (aggregated, business-ready) Delta tables, enabling Change Data Feed on Silver.
46. Re-implement the Silver transformation as a Delta Live Tables pipeline with at least 3 @dlt.expect data quality expectations (mix of warn/drop/fail).
47. Set up Unity Catalog: a 3-level namespace (catalog.schema.table), column-level masking on the customer email column, and confirm automatic lineage is captured from Bronze → Silver → Gold.
48. Create a Delta Share exposing one Gold table as if to an external partner.
49. Orchestrate the whole thing as a multi-task Databricks Workflow: Auto Loader notebook → DLT pipeline → Gold aggregation task, with retries and a failure-alert configured.
50. Stand up a small SQL Warehouse, build a 3–4 tile dashboard on the Gold layer, and add one scheduled alert on a KPI threshold.

Deliverable: Databricks notebooks exported as .py/.sql files into the repo, the DLT pipeline JSON, the Workflow JSON, and a dashboard screenshot.

Phase 9 — Final Integration Report

51. Produce a single architecture diagram showing the full flow: Colab/VS Code → S3 (Bronze/Silver/Gold) → Glue Catalog/Athena → Lake Formation → Redshift Serverless AND Databricks (parallel consumption).
52. Write a 1–2 page cost review: what you provisioned, what you tore down, and what it would cost to run this pipeline monthly at 10x the data volume.
53. Write a short reflection: one design decision you'd change if this were a real production system, and why.

Deliverable: FINAL_REPORT.pdf containing the diagram, cost review, and reflection.

4. Full Deliverables Checklist

#	Deliverable	Phase
1	Git repo retailflow-capstone with README + ARCHITECTURE.md	0/3
2	01_data_generation.ipynb + data_profile_report.md + 4 PNG charts	1
3	s3_ingest/ Boto3 package + upload_day1.py + logs	2
4	S3 bucket screenshots (structure, versioning, encryption, lifecycle)	3
5	athena_cost_comparison.md	4
6	lake_formation_governance.md + access-proof screenshots	5
7	glue_etl_job.py + DQDL ruleset + bookmarks_proof.md + CloudWatch screenshot	6
8	redshift_ddl.sql + redshift_load.sql + redshift_governance.sql + redshift_performance.md	7
9	Databricks notebooks + DLT pipeline JSON + Workflow JSON + dashboard screenshot	8
10	FINAL_REPORT.pdf	9

5. Evaluation Rubric

Category	Weight	What's assessed
Python & Ingestion Quality	15%	Boto3 utility design, retry/logging, code cleanliness
Data Lake Design	20%	Zone structure, partitioning, security, lifecycle policy correctness
Governance	10%	Lake Formation LF-Tags and Unity Catalog access control correctness
Glue ETL & Data Quality	25%	Bookmark correctness, schema evolution handling, DQDL rule coverage, quarantine routing
Redshift Track	17%	Schema design, load correctness, Spectrum, WLM/QMR, masking, performance tuning evidence
Databricks Track	13%	Auto Loader, DLT expectations, Unity Catalog, Workflow orchestration, dashboard